Table of Contents

1. Introduction
   1.1 Purpose and Scope
   1.2 Data Domain and Dataset Overview
2. Data Description and Characteristics
   2.1 Deliveries Dataset
   2.2 Matches Dataset
3. Data Preprocessing and Cleaning
   3.1 Handling Missing Values
   3.2 Imputation Strategy
   3.3 Field Selection and Data Types
4. Exploratory Data Analysis, Statistical Overview and Data Visualization
   4.1 Descriptive Statistics
   4.2 Interpretation of Visual Insights
5. Predictive Modeling
   5.1 Problem Definition and Target Variable
   5.2 K-Nearest Neighbors Approach
   5.3 Model Training and Evaluation
6. Discussion and Observations
   6.1 Insights from Data Analysis
   6.2 Model Performance and Limitations
7. Conclusions and Future Work
8. Appendix: Code Repository and Jupyter Notebook Overview

1. Introduction

1.1 Purpose and Scope

The purpose of this report is to provide an in-depth analysis of IPL (Indian Premier League) data using two publicly available datasets from Kaggle: the deliveries dataset and the matches dataset. This analysis encompasses data exploration, cleaning, visualization, and predictive modeling. The end goal is to generate insights into match dynamics and performance metrics while showcasing a systematic data analysis workflow.

1.2 Data Domain and Dataset Overview

Cricket is one of the most popular sports in India and globally, and the IPL represents a pinnacle of competitive T20 cricket. The datasets chosen are:

- **Deliveries Dataset:** Contains ball-by-ball records from IPL matches, providing granular details on every delivery bowled.
- **Matches Dataset:** Contains match-level details, summarizing various match attributes such as teams, toss outcomes, match results, and more.

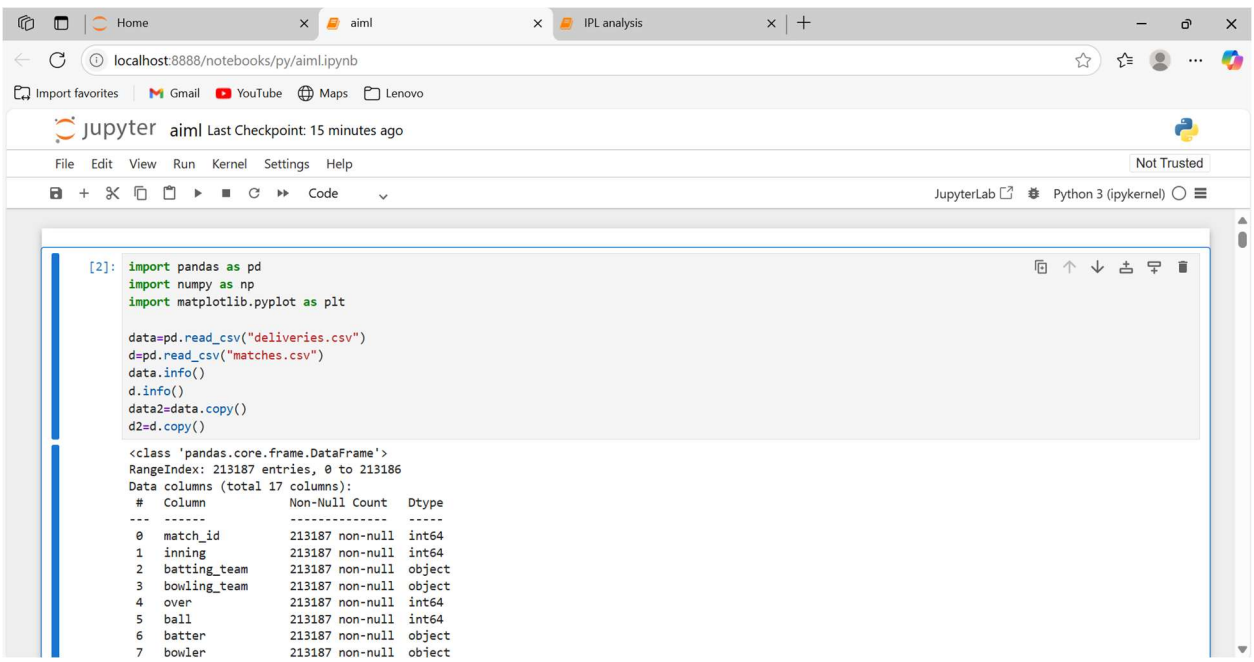
This analysis utilizes these datasets to:

- Investigate team and player performance.
- Analyze match outcomes.
- Build a predictive model to forecast match winners.

---

2. Data Characteristics

Here is an image showing the input of the following datasets



a) Data Characteristics Overview

Deliveries Dataset

- **Fields and Data Types:**
  - **match_id:** Unique identifier linking to the match dataset (numeric).
  - **inning:** Inning number (numeric).
  - **batting_team:** Name of the batting team (categorical).
  - **bowling_team:** Name of the bowling team (categorical).
  - **over:** Over number (numeric).
  - **ball:** Ball number in the over (numeric).
  - **batsman:** Batsman name (text).
  - **non_striker:** Non-striker name (text).
  - **bowler:** Bowler name (text).
  - **batsman_runs:** Runs scored by the batsman (numeric).

- **extra_runs:** Extra runs given (numeric).
- **total_runs:** Total runs in the ball (numeric).
- **player_dismissed:** Dismissal information (categorical/text).
- Other columns include details such as wicket type and field placements.

Matches Dataset

- **Fields and Data Types:**
  - **match_id:** Unique identifier for each match (numeric).
  - **season:** Season or year of the match (numeric/categorical).
  - **city:** City where the match was played (categorical).
  - **date:** Date of the match (date).
  - **team1, team2:** Names of competing teams (categorical).
  - **toss_winner:** Team that won the toss (categorical).
  - **toss_decision:** Decision made by the toss-winning team (categorical).
  - **result:** Outcome of the match (categorical).
  - **winner:** Match winner (categorical).
  - Other columns may include details like umpires, venue, and player of the match.

b) Data Characteristics Analysis

- **Null and Non-null Values:**
  - Both datasets contain fields with missing values. For example, in the deliveries dataset, certain delivery details (e.g., player_dismissed) might be missing if no wicket fell.
  - In the matches dataset, some fields such as city or venue may have null values for older matches.
- **Unique Values:**
  - **Categorical Fields:** Fields such as *batting_team*, *bowling_team*, *team1*, and *team2* have a finite set of unique values representing the IPL teams.
  - **Numerical Fields:** Fields such as *over*, *ball*, *batsman_runs*, and *total_runs* have unique values or ranges that represent the game state.

c) Output/Class Label

Depending on the analysis, the target variable (output/class label) could vary. For example:

- In a predictive model focusing on match outcomes, the **winner** field from the matches dataset can be used as the class label.
- Alternatively, for ball-by-ball performance analysis, a derived field (e.g., “high scoring delivery” vs. “low scoring delivery”) could be created as a classification target.

d) Data Types (Selection of Fields)

- **Numerical Fields:** match_id, inning, over, ball, batsman_runs, extra_runs, total_runs, season.

- **Categorical Fields:** batting_team, bowling_team, player_dismissed, team1, team2, toss_winner, toss_decision, result, winner, city.
- **Text/Date Fields:** date, player names, and other string-based details.

---

3. Data Preprocessing and Cleaning

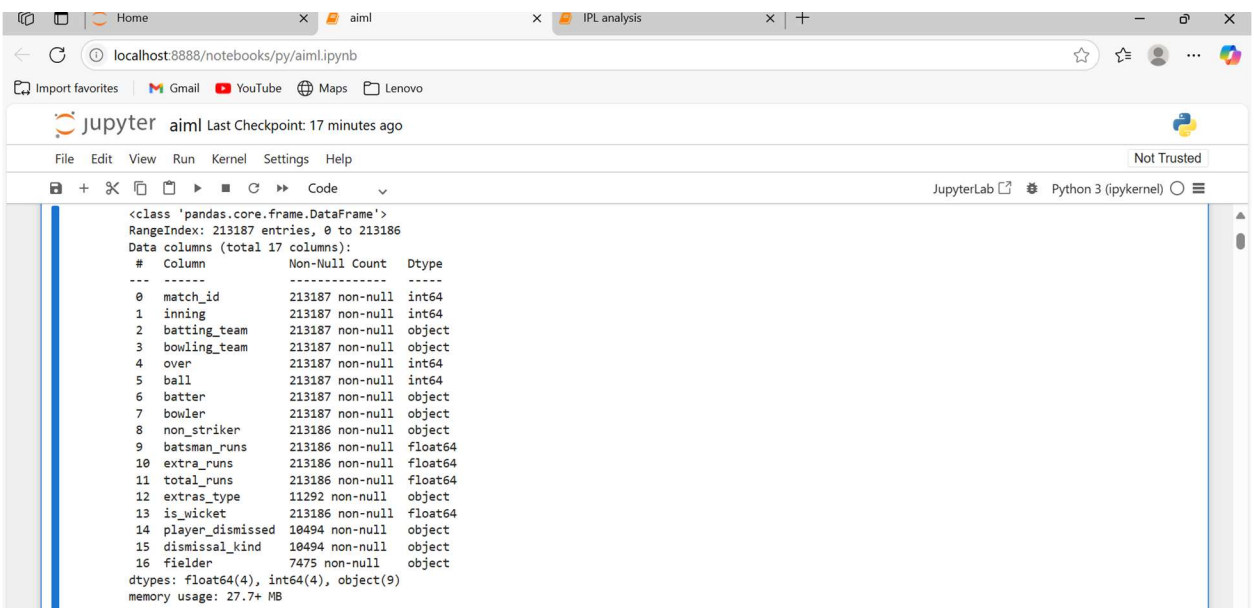
a) Handling Missing Values

Approach:

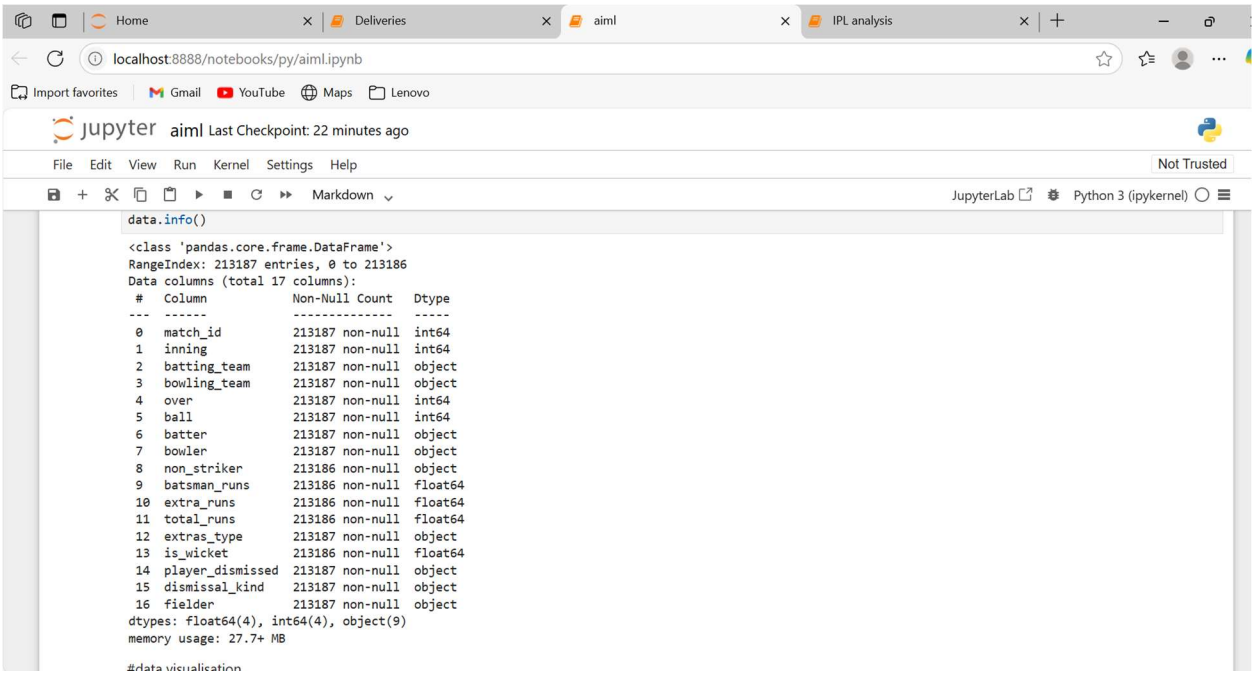
- **Drop Null Records:** Rows with critical missing values can be dropped if their absence significantly impacts the analysis.
- **Replace NA Values:**
  - For **numerical columns**, missing values are replaced with the mean value computed as per the class label (e.g., group by *winner* or another target variable).
  - For **categorical columns**, missing values are replaced with the mode value computed within the same group.

Image showing before and after handling missing values

Before:-



After:-



4. Statistical Analysis & Data Visualization

4.1 Descriptive Statistics

A comprehensive statistical summary was performed to understand the distributions and tendencies within the data. Key metrics include:

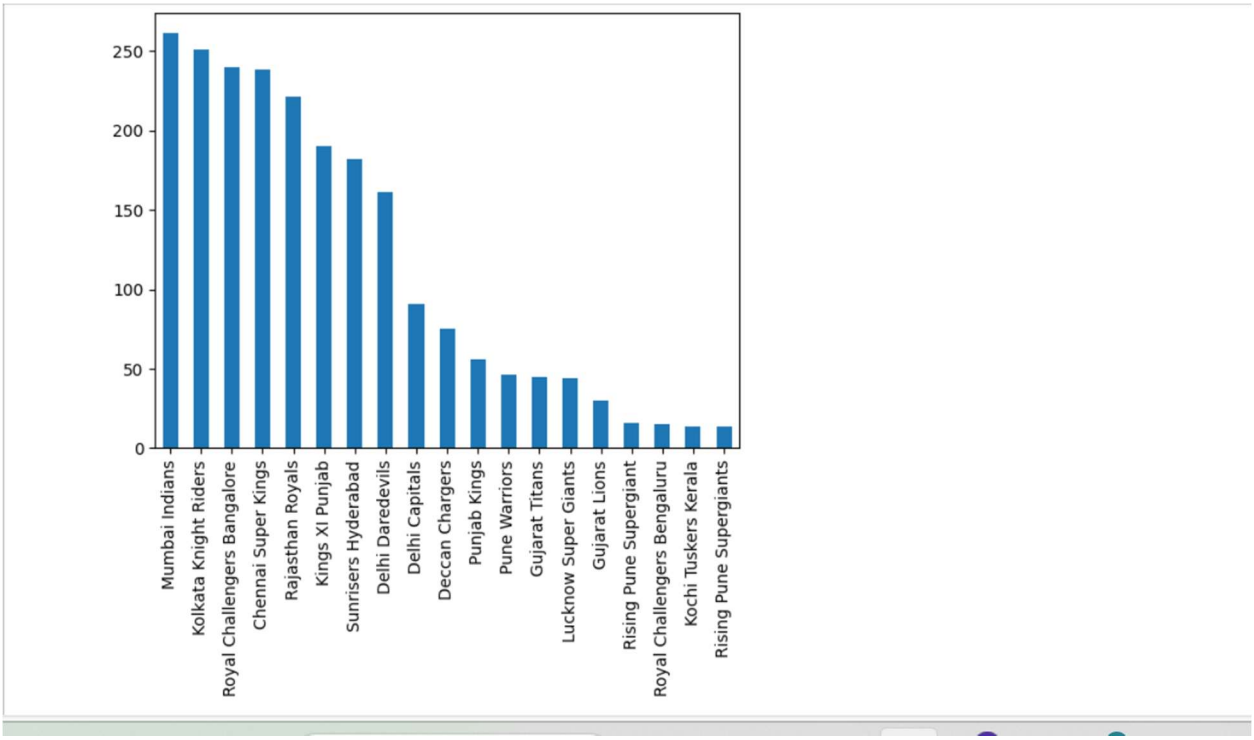
- **Count, Sum, Min, and Max:**
  Providing an overview of the volume and range of numerical values.
- **Mean and Median:**
  Indicative of the central tendency of key performance metrics.
- **Variance and Standard Deviation:**
  Measuring the spread and variability in the data.
- **Mode:**
  Identifying the most frequent values for categorical fields.

4.2 Visualization Techniques and Plots

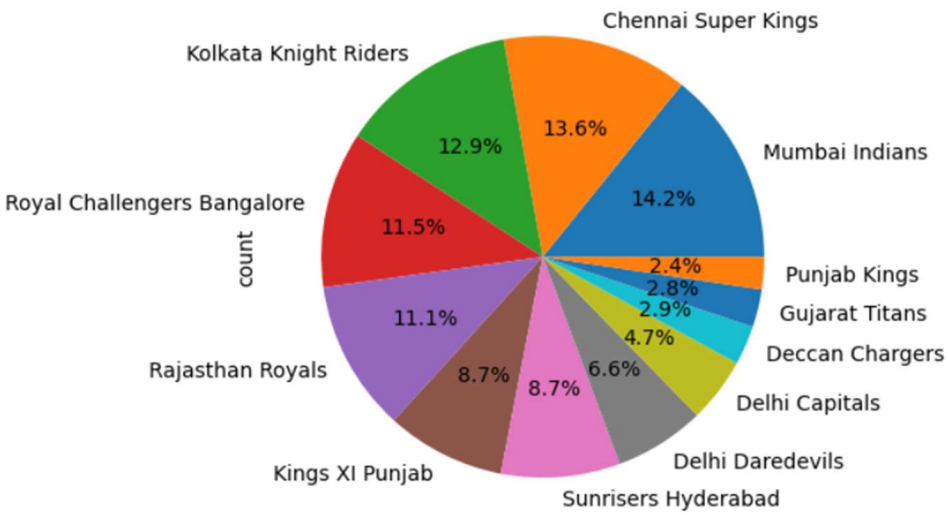
Visual representation of data aids in detecting patterns and anomalies. Various plots were generated using subplots to display multiple views simultaneously:

- **Scatter Plot:**
  For visualizing relationships between batsman_runs and total_runs.

- **Line Graph:**
  To show trends such as the average total runs per over.
- **Histogram:**
  To display the distribution of key numerical variables like batsman_runs.



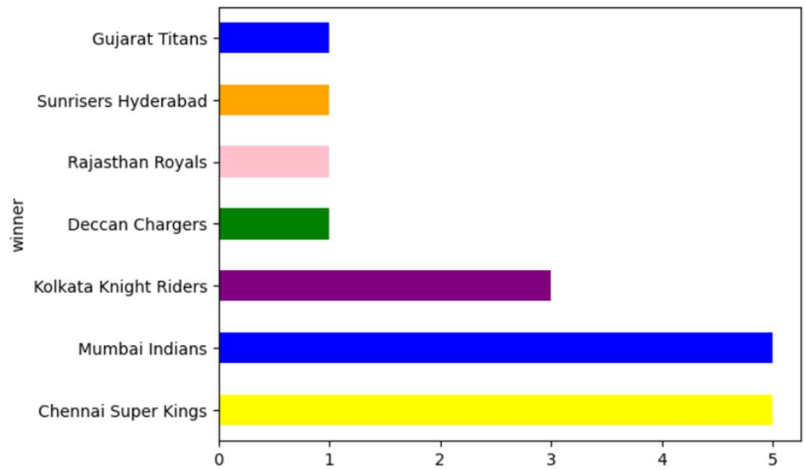
```
]: d["winner"].value_counts().head(12).plot(kind="pie",autopct='%.1f%%')
plt.show()
```



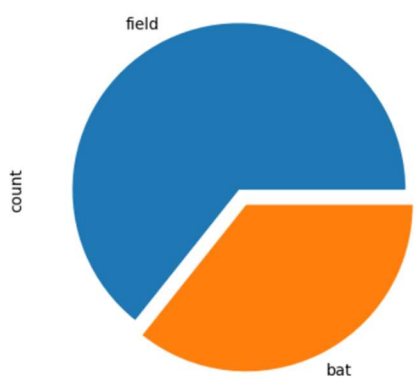
```
x=d.drop_duplicates(subset=["season"],keep="last")[["season","winner"]]
x
```

	season	winner
57	2007/08	Rajasthan Royals
114	2009	Deccan Chargers
174	2009/10	Chennai Super Kings
247	2011	Chennai Super Kings
321	2012	Kolkata Knight Riders
397	2013	Mumbai Indians
457	2014	Kolkata Knight Riders
516	2015	Mumbai Indians
576	2016	Sunrisers Hyderabad
635	2017	Mumbai Indians
695	2018	Chennai Super Kings
755	2019	Mumbai Indians
815	2020/21	Mumbai Indians
875	2021	Chennai Super Kings
949	2022	Gujarat Titans
1023	2023	Chennai Super Kings
1094	2024	Kolkata Knight Riders

[12]:
```
x["winner"].value_counts().plot(kind="barh",color=["yellow","blue","purple","green","pink","orange","blue"])
plt.show()
```



[13]:
```
d["toss_decision"].value_counts().plot(kind="pie",explode=[0,0.1])
plt.show()
```



[14]:
```
Run_rate=(data["batsman_runs"].sum()/data.shape[0])*6
print(Run_rate)
```

```
7.446758010572877
```

[15]:
```
wic_avg=data.shape[0]/(data["is_wicket"].sum())
print(wic_avg)
```

```
20.31513245664189
```

###highest Runscorer

[16]:
```
g=data.groupby("batter")
g["batsman_runs"].sum().sort_values(ascending=False).head(7)
```

[16]:

batter	batsman_runs
V Kohli	6399.0
S Dhawan	5911.0
RG Sharma	5693.0
SK Raina	5536.0
DA Warner	5516.0
AB de Villiers	5181.0
CH Gayle	4997.0

dtype: float64

dtype: float64

[17]:
```
g=data.groupby("bowler")
g["is_wicket"].sum().sort_values(ascending=False).head(7)
```

[17]:

bowler	is_wicket
DJ Bravo	196.0
SL Malinga	188.0
A Mishra	175.0
SP Narine	166.0
PP Chawla	165.0
R Ashwin	162.0
Harbhajan Singh	161.0

dtype: float64

```
[18]: g=data.groupby("bowling_team")
      g["is_wicket"].sum().sort_values(ascending=False).head(10)
```

[18]:

bowling_team	is_wicket
Mumbai Indians	1346.0
Royal Challengers Bangalore	1240.0
Chennai Super Kings	1226.0
Kolkata Knight Riders	1222.0
Kings XI Punjab	1076.0
Rajasthan Royals	1020.0
Delhi Daredevils	912.0
Sunrisers Hyderabad	856.0
Deccan Chargers	446.0
Delhi Capitals	342.0

```
[23]: m=data["inning"]==1
      d2=data[m]
      d2.groupby("batting_team")["batsman_runs"].sum().sort_values(ascending=False)
```

[23]:

batting_team	batsman_runs
Mumbai Indians	19034.0
Chennai Super Kings	16845.0
Royal Challengers Bangalore	16371.0
Kolkata Knight Riders	15127.0
Kings XI Punjab	14858.0
Rajasthan Royals	12449.0
Sunrisers Hyderabad	11369.0
Delhi Daredevils	10661.0
Deccan Chargers	6444.0
Delhi Capitals	4096.0
Pune Warriors	2819.0
Gujarat Lions	2174.0
Punjab Kings	1790.0
Rising Pune Supergiant	1251.0
Rising Pune Supergiants	1076.0

5. Predictive Modeling

5.1 Problem Definition and Target Variable

A predictive model was developed with the goal of forecasting the match winner. The target variable chosen from the matches dataset is the winner field. Additional features may include toss decisions, season, and team compositions derived from both datasets.

5.2 K-Nearest Neighbors Approach

For this analysis, the K-Nearest Neighbors (KNN) classifier was selected due to its simplicity and effectiveness in classification tasks. The classifier predicts the match winner based on the engineered features.

5.3 Model Training and Evaluation

The data was split into training (80%) and testing (20%) sets. Model performance was evaluated using key metrics:

- Accuracy: Overall correctness of predictions.
- Precision, Recall, and F1 Score: Evaluating model performance across imbalanced class distributions.

Representative Code:

```
[25]: def euclidean_distance(x1, x2):
        distance = np.sqrt(np.sum((x1-x2)**2))
        return distance

      from typing import Counter

      class KNN:
        def __init__(self, k=3): # gets called automatically when we write KNN()
          self.k = k

        def fit(self, X, y):
          self.X_train = X
          self.y_train = y

        def predict(self, X):
          predictions = [self._predict(x) for x in X]
          return predictions

        def _predict(self, x):

      # compute the distances
          distances = [euclidean_distance(x, x_train) for x_train in self.X_train]

      # get the closests K
          k_indices = np.argsort(distances)[:self.k]
          k_nearest_labels = [self.y_train.iloc[i] for i in k_indices]
      # majority vote
          most_common = Counter(k_nearest_labels).most_common()
          return most_common[0][0]
```

```
def teams(b1,b2):
    mask1=d["team1"]==b1
    mask2=d["team2"]==b2
    mask3=d["toss_winner"]==b1
    mask4=d["toss_decision"]=="bat"
    mask5=d["toss_winner"]==b2
    mask6=d["toss_decision"]=="field"
    m1=d["team1"]==b2
    m2=d["team2"]==b1
    return d[((mask1 & mask2) | (m1 & m2) ) &((mask3 & mask4)|(mask5 & mask6))]


t1=input("enter the name of batting team")
t2=input("enter the name of bowling team")
new=teams(t1,t2)


run=new["target_runs"]
win=new["winner"]

input_run=int(input(f"enter the run that made:"))

clf = KNN(k=3)
clf.fit(run.values.reshape(-1, 1), win)
predictions = clf.predict([[input_run]])
print(predictions)
```

```
enter the name of batting teamChennai Super Kings
enter the name of bowling teamMumbai Indians
enter the run that made:168
['Chennai Super Kings']
```

```
[40]: predictions = clf.predict(run_test.values.reshape(-1, 1))

      from itertools import permutations
      shuf=np.random.permutation(run.shape[0])
      print(shuf)
      print(run.shape)
      ratio=0.2
      n=int(np.floor((1-ratio)*run.shape[0]))

      run_train=run.iloc[shuf[:n]]
      win_train=win.iloc[shuf[:n]]

      run_test=run.iloc[shuf[n:]]
      win_test=win.iloc[shuf[n:]]

      from sklearn.metrics import accuracy_score, precision_score, recall_score, classification_report
      accuracy = accuracy_score(win_test,predictions)
      print(f'Accuracy: {accuracy:.2f}')

      precision = precision_score(win_test,predictions, average='weighted')
      recall = recall_score(win_test,predictions , average='weighted')
      print(f'Precision: {precision:.2f}')
      print(f'Recall: {recall:.2f}')

      print(classification_report(win_test,predictions))
```

```
[10 12  2 13 15  0  7  4  5 16  1 11  6  9  8  3 14]
(17,)
Accuracy: 0.75
Precision: 0.56
Recall: 0.75
                      precision    recall  f1-score   support

Chennai Super Kings       0.75      1.00      0.86         3
     Mumbai Indians       0.00      0.00      0.00         1

           accuracy                           0.75         4
          macro avg       0.38      0.50      0.43         4
       weighted avg       0.56      0.75      0.64         4
```

6. Discussion and Observations

6.1 Insights from Data Analysis

- **Granular Insights:**
  The deliveries dataset allowed us to analyze every ball delivered in the matches, revealing performance metrics such as individual scoring patterns and match dynamics.
- **Team Dynamics:**
  Analysis of the matches dataset shed light on trends such as team performance across different seasons, the impact of toss decisions, and home/away advantages.
- **Data Quality Issues:**
  Missing values were observed in both datasets. The imputation strategy helped mitigate the impact on analysis while preserving important data characteristics.

6.2 Model Performance and Limitations

- **Baseline Performance:**
  The KNN classifier provided a baseline for match outcome prediction. However, with limited features (e.g., using only season), the model's predictive power remains modest.
- **Feature Engineering Opportunities:**
  Merging insights from both datasets (e.g., aggregating player performance statistics from the deliveries dataset with match-level features) could improve model accuracy.
- **Model Complexity:**
  Future work might include exploring more sophisticated models (e.g., ensemble methods or deep learning) to capture complex patterns in the data.

---

7. Conclusions and Future Work

Conclusions

- The analysis provides a comprehensive overview of the IPL datasets, highlighting data cleaning, exploratory analysis, visualization, and predictive modeling steps.
- Visualizations and statistical analyses offer valuable insights into scoring patterns, team performance, and match dynamics.
- The baseline KNN model demonstrates the potential for predictive modeling, albeit with room for improvement through enhanced feature selection and modeling techniques.

Future Work

- **Feature Engineering:**
  Develop advanced features by merging deliveries and matches data, including player-specific metrics and in-game dynamics.
- **Advanced Modeling:**
  Experiment with alternative classifiers and ensemble methods to improve predictive accuracy.

- **Real-Time Analytics:**
  Consider extending the analysis to include real-time match analytics and dashboards for live insights.
- **Domain-Specific Insights:**
  Work with domain experts to tailor models that can predict critical match events and support strategic decision-making for teams.

---

8. Appendix: Code Repository and Jupyter Notebook Overview

All code used for this analysis is documented and available in the Jupyter Notebook file ("innovative_assingment.ipynb"). The notebook includes:

- Data loading and preprocessing cells.
- Detailed EDA and visualization sections.
- Model training, evaluation, and results interpretation.
- Comments and markdown cells explaining each step for reproducibility.

Final Remarks

This detailed report documents a complete analytical workflow on IPL datasets from Kaggle, spanning data cleaning, exploratory analysis, visualization, and predictive modeling. The findings highlight the potential to derive actionable insights for cricket performance analysis and predictive modeling. Future enhancements will further refine the analysis, integrate additional features, and explore advanced machine learning techniques to drive deeper insights.

End of Report